LAPORAN PRAKTIKUM
ALGORITMA DAN STRUKTUR DATA

Dosen Pengampu: Muhammad Reza Zulman,S.ST.,M.Sc

Di susun oleh :

Nama	: Wahyu Hakim
NIM	2025573010096
Kelas	: TI. 1C
Mata Kuliah	: Algoritma dan Struktur Data

PROGRAM STUDI INFORMATIKA JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER POLITEKNIK NEGERI LHOKSEUMAWE
2026/2027

LEMBAR PENGESAHAN

Nama Praktikum	:	*Benchmark* dan Analisis Kompleksitas Algoritma
Nama	:	Wahyu Hakim
NIM	:	2025573010096
Kelas	:	TI 1C
Jurusan	:	Teknologi Informasi dan Komputer
Prodi	:	Teknik Informatika
Tanggal Praktikum	:	22 April 2026
Tanggal Penyerahan Laporan	:	28 April 2026
Nilai	:	
Keterangan	:	

Buket Rata, 29 April 2026

Muhammad Reza Zulman, S.ST.,M.Sc

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh

Puji syukur saya panjatkan kehadirat Tuhan Yang Maha Esa yang telah memberikan rahmat dan karunia-Nya sehingga saya dapat menyelesaikan laporan "Analisis Kompleksitas Algoritma Menggunakan Big O". Laporan ini saya susun secara sistematis sebagai bentuk pemenuhan tugas mata kuliah Algoritma dan Struktur data.

Dengan terselesainya laporan ini, maka tidak lupa saya mengucapkan banyak terima kasih kepada semua pihak yang terlibat dalam penyusunan laporan ini, khususnya kepada Bapak Muhammad Reza Zulman, S.ST.,M.Sc. selaku Dosen Pengampu.

Demikian laporan yang saya buat, mohon kritik dan sarannya atas kekurangan dalam penyusunan laporan ini. Semoga laporan ini dapat bermanfaat bagi semua pihak dan bagi saya selaku penulis.

Lhokseumawe, 14 Maret 2026

Wahyu Hakim

DAFTAR ISI

LEMBAR PENGESAHAN ..... **i**

KATA PENGANTAR ..... **ii**

DAFTAR ISI ..... **iii**

DAFTAR ISI ..... **iv**

BAB I: PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Rumusan Masalah 1

1.3 Tujuan Praktikum 2

BAB II: DASAR TEORI **3**

2.1 Kompleksitas Algoritma 3

2.2 Kompleksitas Waktu 3

2.3 Kompleksitas Ruang 4

2.4 Notasi Big O 5

BAB III: METODE PRAKTIKUM **6**

3.1 Waktu dan Tempat 6

3.2 Alat dan Bahan 6

3.3 Prosedur Praktikum 6

BAB IV: HASIL DAN PEMBAHASAN **34**

4.1 Hasil praktikum 34

4.2 Pembahasan 35

BAB V: PENUTUP **36**
5.1 Kesimpulan..... 36
5.2 Saran..... 36

DAFTAR PUSTAKA **37**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi menuntut program yang cepat dan efisien. Pemilihan algoritma menjadi faktor penting karena mempengaruhi kinerja sistem saat memproses data dalam jumlah besar. Algoritma yang tidak efisien dapat menyebakan waktu eksekusi lebih lama dan penggunaan memori yang berlebihan.

Analisis kompleksitas algoritma digunakan untuk mengukur efisiensi suatu program. Kompleksitas waktu menunjukkan lama eksekusi berdasarkan ukuran input, sedangkan kompleksitas ruang menunjukkan jumlah memori yang digunakan selama proses berlangsung. Konsep ini dinyatakan dengan notasi Big O.

Dalam praktikum ini dilakukan implementasi beberapa fungsi dengan tingkat kompleksitas yang berbeda seperti $O(1)$, $O(n)$, $O(n \log n)$, dan $O(n^2)$. Setiap fungsi diuji dengan berbagai ukuran data untuk melihat perbedaan waktu eksekusi secara langsung.

Melalui praktikum ini, kamu dapat memahami bagaimana struktur algoritma mempengaruhi performa program. Hasil pengujian membantu dalam memilih algoritma yang sesuai dengan kebutuhan, terutama saat menangani data dalam jumlah besar.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, Maka rumusan masalah dalam Laporan ini adalah sebagai berikut:

1. Bagaimana cara menentukan kompleksitas waktu pada suatu algoritma menggunakan notasi Big O?
2. Bagaimana cara menentukan kompleksitas ruang pada suatu algoritma?
3. Bagaimana perbedaan performa algoritma dengan kompleksitas O(1), O(n), O(n log n), dan $O(n^2)$?
4. Bagaimana pengaruh ukuran input terhadap waktu eksekusi program?
5. Bagaimana hasil pengukuran waktu eksekusi menggunakan Date.now() pada berbagai algoritma?
6. Algoritma mana yang lebih efisien untuk digunakan pada data berukuran besar?

1.3 Tujuan Pembelajaran

Berdasarkan kegiatan praktikum yang telah dilakukan, maka tujuan dari praktikum ini adalah:

1. Memahami konsep kompleksitas algoritma menggunakan notasi Big O
2. Menentukan kompleksitas waktu dan ruang dari berbagai fungsi
3. Mengimplementasikan algoritma dengan kompleksitas O(1), O(n), O(n log n), dan $O(n^2)$
4. Mengukur waktu eksekusi program menggunakan Date.now()
5. Membandingkan performa algoritma berdasarkan ukuran data
1. Menentukan algoritma yang lebih efisien untuk digunakan pada kasus tertentu

BAB II
DASAR TEORI

2.1 Kompleksitas Algoritma

Perkembangan teknologi menuntut program yang cepat dan efisien. Pemilihan algoritma menjadi faktor penting karena mempengaruhi kinerja sistem saat memproses data dalam jumlah besar. Algoritma yang tidak efisien dapat menyebakan waktu eksekusi lebih lama dan penggunaan memori yang berlebihan.

Analisis kompleksitas algoritma digunakan untuk mengukur efisiensi suatu program. Kompleksitas waktu menunjukkan lama eksekusi berdasarkan ukuran input, sedangkan kompleksitas ruang menunjukkan jumlah memori yang digunakan selama proses berlangsung. Konsep ini dinyatakan dengan notasi Big O.

Dalam praktikum ini dilakukan implementasi beberapa fungsi dengan tingkat kompleksitas yang berbeda seperti $O(1)$, $O(n)$, $O(n \log n)$, dan $O(n^2)$. Setiap fungsi diuji dengan berbagai ukuran data untuk melihat perbedaan waktu eksekusi secara langsung.

Melalui praktikum ini, kamu dapat memahami bagaimana struktur algoritma mempengaruhi performa program. Hasil pengujian membantu dalam memilih algoritma yang sesuai dengan kebutuhan, terutama saat menangani data dalam jumlah besar.

2.2 Kompleksitas Waktu

Array adalah struktur data yang digunakan untuk menyimpan beberapa nilai dalam satu variabel. Setiap elemen memiliki indeks yang dimulai dari 0, sehingga memudahkan proses akses dan pengolahan data. Array dapat berisi berbagai jenis data seperti angka, *string*, dan *object.*

Dalam JavaScript, array memiliki berbagai method untuk mengelola data. *Method push* dan *pop* digunakan untuk menambah dan menghapus elemen di akhir array. *Method unshift* dan *shift* digunakan untuk manipulasi di bagian awal array. Selain itu, terdapat *method* seperti *forEach,* map, dan filter yang digunakan untuk mengolah data dengan lebih efisien.

2.3 Kompleksitas Ruang

Kompleksitas ruang adalah ukuran jumlah memori tambahan yang digunakan oleh algoritma selama proses eksekusi. Yang dihitung adalah memori di luar input utama, seperti variabel, array baru, atau struktur data lain.

Notasi yang digunakan juga Big O. Nilainya menunjukkan seberapa besar memori bertambah saat ukuran input meningkat.

O(1) berarti hanya menggunakan memori tetap, misalnya hanya beberapa variabel. O(n) berarti memori bertambah seiring jumlah data, biasanya karena membuat array baru atau menggunakan struktur seperti Set.

Rekursi juga mempengaruhi kompleksitas ruang karena setiap pemanggilan fungsi disimpan dalam call stack. Semakin dalam rekursi, semakin besar memori yang digunakan.

Dalam praktikum ini terlihat bahwa algoritma yang menggunakan struktur

tambahan atau rekursi cenderung membutuhkan memori lebih besar dibandingkan algoritma yang menggunakan pendekatan iteratif sederhana.

2.4 Notasi Big O

Notasi Big O adalah cara untuk menyatakan kompleksitas algoritma berdasarkan pertumbuhan kinerja terhadap ukuran input. Notasi ini tidak fokus pada waktu nyata, tetapi pada pola pertumbuhan saat data semakin besar.

Big O digunakan untuk membandingkan efisiensi algoritma. Dengan notasi ini, kamu bisa mengetahui algoritma mana yang lebih cepat atau lebih hemat memori tanpa harus menjalankan program.

Beberapa contoh notasi Big O yang umum yaitu O(1), O(n), O(log n), O(n log n), dan $O(n^2)$. O(1) menunjukkan performa paling stabil, sedangkan $O(n^2)$ menunjukkan pertumbuhan yang sangat cepat dan kurang efisien untuk data besar.

Dalam praktikum ini, notasi Big O digunakan untuk menganalisis setiap fungsi yang dibuat. Hasil analisis membantu dalam memahami perbedaan performa dan memilih algoritma yang lebih tepat sesuai kebutuhan.

BAB III
METODELOGI PRAKTIKUM

3.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada tanggal 8 bertempat di laboratoriumgedung jurusan Teknologi Informasi Komputer, Ruang 106. Kegiatan praktikum dilakukan sesuai dengan jadwal yang telah ditentukan.

3.2 Alat dan Bahan

Alat:

- Laptop atau computer
- Visual Studio Code atau text editor lainnya
- Node.js

Bahan:

- Modul praktikum Algoritma dan Struktur Data
- Data percobaan untuk pengujian program
- Mengukur waktu eksekusi dengan Date.now()
- Menjalankan program menggunakan Node.js
- • Menganalisis hasil

3.3 Prosedur Praktikum

Langkah langkah yang dilakukan pada praktikum adalah sebagai berikut.

1. Menyiapkan alat dan bahan yang dibutuhkan laptop, text editor, dan

2. Membuka Git Bash kemudian masuk ke repositary 2025573010096_Algoritma_Dan_Struktur_Data menggunakan perintah cd.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~
$ cd 2025573010096_Algoritma_Dan_Struktur_Data

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data (
main)
$
```

3. Membuat folder baru bernama praktikum-5 menggunakan perintah mkdir praktikum-5, kemudian masuk ke dalam folder tersebut dengan perintah cd praktikum-5

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data (main)
$ mkdir praktikum-5

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data (main)
$ cd praktikum-5
```

4. Inisialisasi proyek Node.js dengan menjalankan perintah npm init -y untuk membuat file package.json secara otomatis.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ npm init -y
Wrote to C:\Users\Lenovo LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

5. Membuat file

Pastikan berada di dalam folder praktikum-5. Kemudian buat file baru dengan nama 01-intro-kompleksitas.js.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$
```

6. Menuliskan dan memahami kode

Kode yang ditulis berfungsi untuk membandingkan performa beberapa algoritma berdasarkan kompleksitas waktu. Terdapat tiga fungsi utama yaitu jumlahkanRumus dengan kompleksitas O(1), jumlahkanLinear dengan kompleksitas O(n), dan cariPasangan dengan kompleksitas O(n^2). Program juga menggunakan fungsi ukurWaktu untuk menghitung lama eksekusi setiap algoritma menggunakan Date.now(). Dengan kode ini, kamu dapat melihat perbedaan kecepatan antara algoritma yang efisien dan yang kurang efisien.

```
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris

// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}

const N = 100_000; // ukuran data: 100 ribu

// --- Algoritma A: O(n) — loop satu kali ---
```

```
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}

// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}

// --- Algoritma C: O(n2) — loop bersarang ---
function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i],
arr[j]]);
    }
  }
  return pasangan;
}

const data = Array.from({ length: 5000 }, (_, i) => i);

console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("0(1)   jumlahkanRumus  ", () => jumlahkanRumus(N));
ukurWaktu("0(n)   jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("0(n2)  cariPasangan    ", () => cariPasangan(data));
console.log("\nHasil sama?", jumlahkanLinear(100) ===
jumlahkanRumus(100));
```

7. Menjalankan dan melihat output

Simpan file, lalu jalankan program menggunakan perintah node 01-intro-kompleksitas.js. Output akan menampilkan waktu eksekusi dari masing-masing fungsi di terminal. Hasil menunjukkan bahwa O(1) paling cepat, diikuti O(n), dan O(n^2) paling lambat. Program juga menampilkan pengecekan

bahwa hasil perhitungan fungsi linear dan rumus bernilai sama.

```
=== Perbandingan Waktu Eksekusi ===
0(1)   jumlahkanRumus   : 0 ms
0(n)   jumlahkanLinear : 1 ms
0(n2)  cariPasangan     : 15 ms

Hasil sama? true
```

8. Memastikan Membuat file

   Pastikan berada di dalam folder praktikum-5. Kemudian buat file baru dengan nama 02-destructuring-spread.js.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ touch 02-destructuring-spread.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$
```

9. Menuliskan dan memahami kode

   Tuliskan kode sesuai modul ke dalam file tersebut. Kode ini berfungsi untuk memahami penggunaan destructuring dan spread operator pada JavaScript. Destructuring digunakan untuk mengambil nilai dari array atau object secara langsung ke dalam variabel. Spread operator digunakan untuk menyalin atau menggabungkan array dan object dengan cara yang lebih singkat dan efisien.

   Tambahkan juga latihan dalam file yang sama. Latihan berisi beberapa fungsi dengan kompleksitas berbeda yaitu O(1), O(n^2), O(log n), dan O(n^3). Setiap fungsi dijalankan dengan n = 1000 dan waktu eksekusinya diukur menggunakan Date.now(). Dibuat juga fungsi hitungKompleksitas untuk

membantu pengujian.

```
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O

// ── O(1) — Konstan ──────────────────────────────────────
function ambilPertama(arr) {return arr[0];}              // O(1)
function tambahkanItem(arr, item) {arr.push(item);}      // O(1)
function isGenap(n) {return n % 2 === 0;}                // O(1)

// ── O(log n) — Logaritmik ───────────────────────────────
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah
${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}

// ── O(n) — Linear ───────────────────────────────────────
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}

// ── O(n2) — Kuadratik ───────────────────────────────────
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
```

```
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}

// ─── O(2n) — Eksponensial ──────────────────────────
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2); // dua
panggilan rekursif!
}

// ── Demonstrasi ──────────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() *
1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i)
+ " ");
console.log("");

// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
```

```
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now(); fibMemo(35); console.log(Date.now() - t, "ms");
```

10. Menjalankan dan melihat output

Simpan file, lalu jalankan program menggunakan perintah node 02-destructuring-spread.js. Output akan menampilkan hasil penggunaan destructuring dan spread operator sesuai dengan kode yang dijalankan.

```
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 997

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
74 ms
Waktu fib(35) O(n) memoization:
0 ms
```

11. Membuat file

Buat file baru dengan nama latihan-1.js di dalam folder praktikum-5.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ touch latihan-1.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$
```

12. Menuliskan dan memahami kode

Tuliskan beberapa fungsi untuk dianalisis kompleksitasnya. Fungsi A memiliki kompleksitas O(1), Fungsi B O(n^2), Fungsi C O(log n), dan Fungsi D O(n^3). Tambahkan komentar pada setiap fungsi yang menjelaskan Big O dan alasannya. Implementasikan semua fungsi, lalu jalankan dengan n = 1000. Gunakan Date.now() untuk mengukur waktu eksekusi. Buat juga fungsi hitungKompleksitas(n, fn) untuk menjalankan fungsi dan menampilkan waktu eksekusi.

```
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log("Waktu eksekusi:", end - start, "ms");
}

// Fungsi A
function fungsiA(n) {
  // Big O: O(1)
  // Alasan: hanya 1 operasi, tidak tergantung n
  return n * 2;
}

// Fungsi B
function fungsiB(n) {
  // Big O: O(n²)
```

```
  // Alasan: 2 loop bersarang, masing-masing n
  for (let i = 0; i < n; i++) for (let j = 0; j < n; j++);
}

// Fungsi C
function fungsiC(n) {
  // Big O: O(log n)
  // Alasan: i dikali 2 setiap iterasi
  for (let i = 1; i < n; i *= 2);
}

// Fungsi D
function fungsiD(n) {
  // Big O: O(n³)
  // Alasan: 3 loop bersarang
  const arr = Array.from({ length: n }, (_, i) => i);
  arr.forEach((x) => arr.forEach((y) => arr.forEach((z) => {})));
}

// Jalankan
console.log("Fungsi A (O(1))");
hitungKompleksitas(1000, fungsiA);

console.log("Fungsi B (O(n²))");
hitungKompleksitas(1000, fungsiB);

console.log("Fungsi C (O(log n))");
hitungKompleksitas(1000, fungsiC);

// pakai n kecil biar tidak berat
console.log("Fungsi D (O(n³))");
hitungKompleksitas(100, fungsiD);
```

13. Menjalankan dan melihat output

    Simpan file, lalu jalankan dengan perintah node latihan-1.js. Output akan menampilkan waktu eksekusi dari setiap fungsi sehingga perbedaan kompleksitas dapat terlihat.

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
5\tempCodeRunnerFile.js"
Fungsi A (O(1))
Waktu eksekusi: 0 ms
Fungsi B (O(n²))
Waktu eksekusi: 2 ms
Fungsi C (O(log n))
Waktu eksekusi: 0 ms
Fungsi D (O(n³))
Waktu eksekusi: 2 ms
```

14. Membuat file

Pastikan berada di dalam folder praktikum-5. Kemudian buat file baru dengan nama 03-space-complexity.js.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ touch 03-space-complexity.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$
```

15. Menuliskan dan memahami kode

Tuliskan kode sesuai modul ke dalam file tersebut. Kode ini digunakan untuk memahami kompleksitas ruang pada algoritma. Terdapat beberapa fungsi seperti penggunaan variabel sederhana dengan O(1), pembuatan array baru dengan O(n), serta rekursi yang menggunakan call stack. Kode ini membantu melihat perbedaan penggunaan memori pada setiap pendekat

Tambahkan juga latihan dalam file yang sama. Latihan berisi dua fungsi yaitu cariPasanganLambat dengan kompleksitas O(n^2) menggunakan nested

loop dan cariPasanganCepat dengan kompleksitas O(n) menggunakan Set. Kedua fungsi diuji dengan data kecil dan data besar, lalu waktu eksekusinya diukur menggunakan Date.now() untuk melihat perbedaan performa.

```
// 03-space-complexity.js

// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
}

// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}

// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1); // n frame di call stack
}

// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
  return hasil;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n)
space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
```

```javascript
// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
  const seen = new Set(); // Set tumbuh hingga n elemen
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7

// --- cariPasanganLambat ---
// Time: O(n²)
// Space: O(1)
// Alasan: nested loop, tidak pakai struktur tambahan
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// --- cariPasanganCepat ---
// Time: O(n)
// Space: O(n)
// Alasan: 1 loop + Set untuk lookup cepat
function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (const x of arr) {
    const pasangan = target - x;
    if (seen.has(pasangan)) {
      return [pasangan, x];
    }
    seen.add(x);
  }
  return null;
}
```

```
// --- Test kecil ---
const testArr = [2, 7, 11, 15];
const target = 9;

console.log("Lambat:", cariPasanganLambat(testArr, target));
console.log("Cepat :", cariPasanganCepat(testArr, target));

// --- Benchmark ---
function ukurWaktu(label, fn) {
  const start = Date.now();
  fn();
  const end = Date.now();
  console.log(label, ":", end - start, "ms");
}

// buat array 50.000 angka acak
const dataBesar = Array.from({ length: 50000 }, () =>
  Math.floor(Math.random() * 100000),
);

// target yang kemungkinan tidak ada
const targetBesar = -1;

console.log("\n=== Benchmark ===");

ukurWaktu("Lambat O(n²)", () => cariPasanganLambat(dataBesar,
targetBesar));

ukurWaktu("Cepat  O(n)", () => cariPasanganCepat(dataBesar,
targetBesar));
```

16. Menjalankan dan melihat output

Simpan file, lalu jalankan program menggunakan perintah node 03-space-complexity.js. Output akan menampilkan hasil dari setiap fungsi yang dijalankan.

```
Jumlah array : 55
```

```
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
Lambat: [ 2, 7 ]
Cepat : [ 2, 7 ]

=== Benchmark ===
Lambat O(n²) : 991 ms
Cepat  O(n) : 9 ms
```

17. Membuat file

Pastikan berada di dalam folder praktikum-5. Kemudian buat file baru dengan nama 04-refactor.js.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktik
um-5 (main)
$ touch 04-refactor.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktik
um-5 (main)
$
```

18. Menuliskan dan memahami kode

Tuliskan kode sesuai modul ke dalam file tersebut. Kode ini digunakan untuk melakukan refactor pada beberapa fungsi agar lebih efisien. Perbaikan dilakukan dengan mengubah algoritma yang memiliki kompleksitas tinggi menjadi lebih rendah, sehingga waktu eksekusi menjadi lebih cepat dan penggunaan memori lebih optimal.

```
// 05-refactor.js — Refactoring kode berperforma buruk
```

```
// ═══════════════════════════════════════════════
// SKENARIO 1: Cek duplikat dalam array
// ═══════════════════════════════════════════════

// BURUK: O(n²) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] ===
arr[j]) return true;
  return false;
}

// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

// ═══════════════════════════════════════════════
// SKENARIO 2: Frekuensi kemunculan elemen
// ═══════════════════════════════════════════════

// BURUK: O(n²) — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
```

```
// BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}

// ── Benchmark ──────────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n²):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

19. Menjalankan dan melihat output

Simpan file, lalu jalankan program menggunakan perintah node 04-refactor.js. Output akan menampilkan hasil dari fungsi sebelum dan sesudah dilakukan refactor sehingga perbedaannya dapat terlihat.

```
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

20. Membuat folder baru dengan nama tugas menggunakan perintah mkdir tugas.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ mkdir tugas

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$
```

21. Membuat file

Membuat dua file baru di dalam folder tugas dengan nama tugas-1.js dan tugas-2.js menggunakan perintah touch tugas/tugas-1.js tugas/tugas-2.js.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$
```

22. Menuliskan dan memahami kode

Tuliskan kode sesuai instruksi ke dalam file tugas/tugas-1.js. Kode berisi tiga fungsi yaitu fungsi A untuk mencari intersection dengan dua pendekatan $O(n^2)$ dan $O(n)$ menggunakan Set, fungsi B untuk mengelompokkan anagram dengan memanfaatkan hasil sorting sebagai kunci, dan fungsi C untuk mencari kondisi tertentu dengan dua pendekatan $O(n^3)$ dan $O(n \log n)$. Setiap fungsi dilengkapi dengan komentar kompleksitas waktu dan ruang, serta dilakukan pengukuran waktu eksekusi menggunakan data besar.

```
// ================================
// TUGAS 1
// ================================

// helper ukur waktu
function ukurWaktu(label, fn) {
  const start = Date.now();
  fn();
  const end = Date.now();
  console.log(label, ":", end - start, "ms");
}

// ================================
// FUNGSI A: INTERSECTION
// ================================

// O(n²) Time, O(1) Space
```

```
// Alasan: nested loop, tanpa struktur tambahan
function intersectionLambat(arr1, arr2) {
  const hasil = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !hasil.includes(arr1[i])) {
        hasil.push(arr1[i]);
      }
    }
  }
  return hasil;
}

// O(n) Time, O(n) Space
// Alasan: pakai Set untuk lookup cepat
function intersectionCepat(arr1, arr2) {
  const set2 = new Set(arr2);
  const hasil = [];

  for (const x of arr1) {
    if (set2.has(x) && !hasil.includes(x)) {
      hasil.push(x);
    }
  }
  return hasil;
}

// ================================
// FUNGSI B: ANAGRAM
// ================================

// O(n * k log k) Time, O(n) Space
// n = jumlah kata, k = panjang kata
// Alasan: setiap kata di-sort lalu dimasukkan ke object
function kelompokAnagram(arr) {
  const map = {};

  for (const kata of arr) {
    const key = kata.split("").sort().join("");
    if (!map[key]) map[key] = [];
    map[key].push(kata);
```

```
  }

  return Object.values(map);
}

// ==============================
// FUNGSI C: KONDISI KUADRAT
// ==============================

// O(n³) Time, O(1) Space
// Alasan: 3 loop bersarang
function cekKuadratLambat(arr) {
  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      for (let k = 0; k < n; k++) {
        if (arr[i] + arr[j] === arr[k] * arr[k]) {
          return true;
        }
      }
    }
  }
  return false;
}

// O(n log n) Time, O(n) Space
// Alasan: sort + binary search
function cekKuadratCepat(arr) {
  const sorted = [...arr].sort((a, b) => a - b);

  for (let i = 0; i < sorted.length; i++) {
    for (let j = i + 1; j < sorted.length; j++) {
      const target = sorted[i] + sorted[j];

      // binary search target
      let kiri = 0;
      let kanan = sorted.length - 1;

      while (kiri <= kanan) {
        const mid = Math.floor((kiri + kanan) / 2);
```

```
        const kuadrat = sorted[mid] * sorted[mid];

        if (kuadrat === target) return true;
        if (kuadrat < target) kiri = mid + 1;
        else kanan = mid - 1;
      }
    }
  }
  return false;
}

// ================================
// TEST KECIL
// ================================

console.log("=== TEST KECIL ===");

console.log("Intersection:", intersectionCepat([1, 2, 3], [2, 3,
4]));

console.log(
  "Anagram:",
  kelompokAnagram(["eat", "tea", "tan", "ate", "nat", "bat"]),
);

console.log("Cek Kuadrat:", cekKuadratCepat([2, 3, 5]));

// ================================
// BENCHMARK DATA BESAR
// ================================

console.log("\n=== BENCHMARK ===");

// data besar
const arr1 = Array.from({ length: 10000 }, () =>
  Math.floor(Math.random() * 5000),
);
const arr2 = Array.from({ length: 10000 }, () =>
  Math.floor(Math.random() * 5000),
);
```

```
ukurWaktu("Intersection Lambat O(n²)", () =>
intersectionLambat(arr1, arr2));

ukurWaktu("Intersection Cepat O(n)", () =>
intersectionCepat(arr1, arr2));

// anagram
const words = Array.from({ length: 5000 }, () =>
  Math.random().toString(36).substring(2, 5),
);

ukurWaktu("Anagram", () => kelompokAnagram(words));

// cek kuadrat
const angka = Array.from({ length: 2000 }, () =>
  Math.floor(Math.random() * 100),
);

ukurWaktu("Kuadrat Lambat O(n³)", () => cekKuadratLambat(angka));

ukurWaktu("Kuadrat Cepat O(n log n)", () =>
cekKuadratCepat(angka));
```

23. Menjalankan dan melihat output

Simpan file, lalu jalankan program menggunakan perintah node tugas/tugas-1.js. Output akan menampilkan hasil dari setiap fungsi serta perbandingan waktu eksekusi antara versi algoritma yang berbeda, sehingga terlihat mana yang lebih efisien.

```
=== TEST KECIL ===
Intersection: [ 2, 3 ]
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Cek Kuadrat: false

=== BENCHMARK ===
Intersection Lambat O(n²) : 132 ms
Intersection Cepat O(n) : 13 ms
```

```
Anagram : 10 ms
Kuadrat Lambat O(n³) : 2 ms
Kuadrat Cepat O(n log n) : 1 ms
```

24. Menuliskan dan memahami kode

Tuliskan kode sesuai instruksi ke dalam file tugas/tugas-2.js. Kode berisi empat fungsi dengan kompleksitas berbeda yaitu O(1), O(n), O(n log n), dan O(n^2). Buat fungsi benchmarkSemua(ukuranData) untuk menguji setiap fungsi dengan berbagai ukuran data. Tambahkan juga komentar yang menjelaskan pola pertumbuhan waktu eksekusi berdasarkan hasil pengujian.

```
// tugas-2.js

// =======================
// DEFINISI FUNGSI
// =======================

// O(1)
// Time: O(1) | Space: O(1)
function fn_O1(n) {
  return n + 1;
}

// O(n)
// Time: O(n) | Space: O(1)
function fn_On(n) {
  let total = 0;
  for (let i = 0; i < n; i++) {
    total += i;
  }
  return total;
}

// O(n log n)
// Time: O(n log n) | Space: O(1)
function fn_OnLogn(n) {
```

```
  let total = 0;

  for (let i = 0; i < n; i++) {
    // loop log2(n)
    for (let j = 1; j < n; j *= 2) {
      total += j;
    }
  }

  return total;
}

// O(n^2)
// Time: O(n^2) | Space: O(1)
function fn_On2(n) {
  let total = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      total += i + j;
    }
  }

  return total;
}

// ========================
// FUNGSI UKUR WAKTU
// ========================

function ukurWaktu(fn, n) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  return end - start;
}

// ========================
// BENCHMARK
// ========================
```

```
function benchmarkSemua(ukuranData) {
  for (let i = 0; i < ukuranData.length; i++) {
    const n = ukuranData[i];

    console.log("\nUkuran data:", n);

    console.log("O(1):", ukurWaktu(fn_O1, n), "ms");
    console.log("O(n):", ukurWaktu(fn_On, n), "ms");
    console.log("O(n log n):", ukurWaktu(fn_OnLogn, n), "ms");
    console.log("O(n^2):", ukurWaktu(fn_On2, n), "ms");
  }
}

// ========================
// JALANKAN
// ========================

benchmarkSemua([100, 500, 1000, 5000, 10000]);

/*
Analisis hasil:

1. O(1)
Waktu hampir tidak berubah walaupun n bertambah.

2. O(n)
Waktu naik secara linear.
Jika n naik 10x, waktu juga kira-kira naik 10x.

3. O(n log n)
Lebih lambat dari O(n), tapi jauh lebih cepat dari O(n^2).
Kenaikan tidak terlalu tajam.

4. O(n^2)
Paling lambat.
Saat n besar, waktu naik sangat drastis.

Kesimpulan:
Hasil sesuai teori Big O.
Semakin tinggi kompleksitas, semakin cepat waktu eksekusi
membesar saat n naik.
```

```
*/
```

25. Menjalankan dan melihat output

Simpan file, lalu jalankan program menggunakan perintah node tugas/tugas-2.js. Output akan menampilkan waktu eksekusi dari setiap fungsi untuk setiap ukuran data, sehingga dapat terlihat perbedaan performa sesuai dengan teori Big O.

```
Ukuran data: 100
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 0 ms

Ukuran data: 500
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 1 ms

Ukuran data: 1000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 2 ms

Ukuran data: 5000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 37 ms

Ukuran data: 10000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 137 ms
```

26. Menambahkan seluruh file yang telah dibuat ke staging area menggunakan perintah git add . sehingga semua perubahan siap untuk dilakukan commit.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ git add .
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it
```

27. Melakukan commit terhadap seluruh perubahan yang telah ditambahkan ke staging area menggunakan perintah git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma" untuk menyimpan riwayat perubahan proyek.

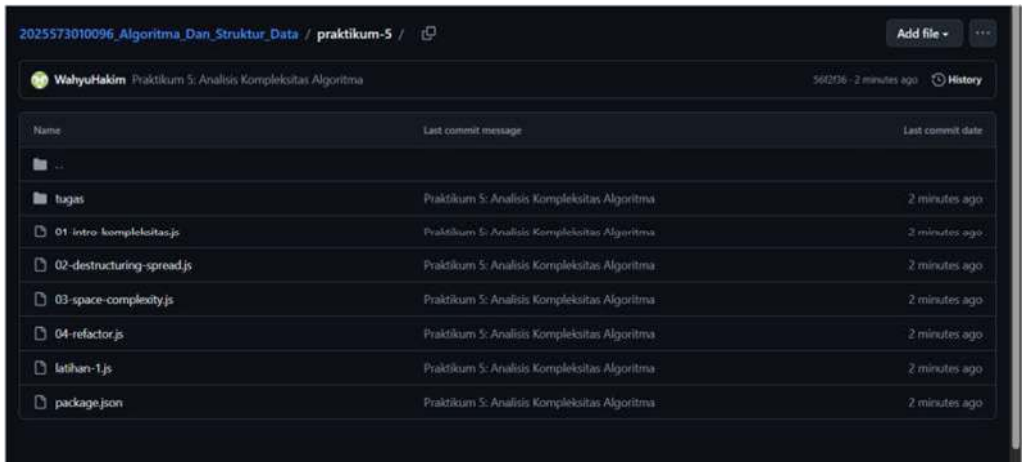
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
```

28. Mengirimkan seluruh perubahan yang telah di-commit ke repository remote menggunakan perintah git push origin main sehingga file dapat tersimpan di GitHub.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-5 (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.59 KiB | 613.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/WahyuHakim/2025573010096_Algoritma_Dan_Struktur_Data.git
   5f786eb..56f2f36  main -> main
```

29. Membuka repository 2025573010096_Algoritma_Dan_Struktur_Data pada GitHub, kemudian masuk ke folder praktikum-5 untuk memastikan seluruh file yang telah di-push sudah berhasil tersimpan dan dapat dilihat.


2025573010096_Algoritma_Dan_Struktur_Data / praktikum-5 /
Add file
WahyuHakim Praktikum 5: Analisis Kompleksitas Algoritma
History
Name
Last commit message
Last commit date
..
tugas
Praktikum 5: Analisis Kompleksitas Algoritma
2 minutes ago
01-intro-kompleksitas.js
02-destructuring-spread.js
03-space-complexity.js
04-refactor.js
latihan-1.js
package.json

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil praktikum

Pada praktikum ini telah dibuat beberapa file program JavaScript yang berisi implementasi algoritma dengan berbagai tingkat kompleksitas. Setiap file dijalankan menggunakan Node.js untuk melihat hasil dan mengukur performa.

Pada file 01-intro-kompleksitas.js, program menampilkan perbandingan waktu eksekusi antara algoritma O(1), O(n), dan $O(n^2)$. Hasil menunjukkan bahwa algoritma dengan kompleksitas O(1) memiliki waktu paling cepat, sedangkan $O(n^2)$ paling lambat.

Pada file 02-destructuring-spread.js, program menampilkan hasil penggunaan destructuring dan spread operator dalam pengolahan data array dan object. Selain itu, ditambahkan latihan yang menguji beberapa fungsi dengan kompleksitas berbeda dan menampilkan waktu eksekusinya.

Pada file 03-space-complexity.js, program menampilkan penggunaan memori dari beberapa fungsi seperti penggunaan variabel sederhana, pembuatan array baru, dan rekursi. Hasil menunjukkan perbedaan penggunaan ruang pada setiap pendekatan.

Pada file 04-space-complexity.js, program menampilkan perbandingan dua metode dalam mencari pasangan angka, yaitu metode lambat dengan nested loop dan metode cepat menggunakan Set. Hasil menunjukkan bahwa metode cepat memiliki waktu eksekusi yang lebih baik.

Pada folder tugas, terdapat dua file yaitu tugas-1.js dan tugas-2.js. File tugas-1.js berisi implementasi dan analisis beberapa fungsi dengan kompleksitas

berbeda, sedangkan tugas-2.js berisi benchmark untuk menguji performa algoritma O(1), O(n), O(n log n), dan O(n^2) dengan berbagai ukuran data.

Berdasarkan hasil pengujian, terlihat bahwa algoritma dengan kompleksitas lebih rendah memiliki performa yang lebih baik, terutama saat ukuran data semakin besar.

4.2 Pembahasan

Hasil praktikum menunjukkan bahwa kompleksitas algoritma sangat mempengaruhi performa program. Algoritma dengan kompleksitas kecil seperti O(1) dan O(n) memiliki waktu eksekusi yang lebih cepat dan stabil, bahkan saat ukuran data meningkat.

Pada pengujian yang dilakukan, algoritma O(1) tidak mengalami perubahan waktu yang signifikan. Algoritma O(n) mengalami peningkatan waktu secara linear sesuai jumlah data. Algoritma O(n^2) mengalami peningkatan waktu yang sangat cepat sehingga kurang efisien untuk data besar.

Penggunaan struktur data juga mempengaruhi performa. Pada kasus pencarian pasangan angka, metode menggunakan Set lebih cepat dibandingkan nested loop. Hal ini karena pencarian data dalam Set dapat dilakukan lebih efisien.

Namun, penggunaan metode yang lebih cepat biasanya membutuhkan memori tambahan. Hal ini terlihat pada penggunaan Set yang memiliki kompleksitas ruang O(n), sedangkan metode nested loop tidak membutuhkan memori tambahan yang besar.

Hasil benchmark pada tugas menunjukkan bahwa pola pertumbuhan waktu eksekusi sesuai dengan teori Big O. Semakin besar kompleksitas, semakin besar peningkatan waktu saat ukuran data bertambah. Oleh karena itu, pemilihan algoritma yang tepat sangat penting untuk menghasilkan program yang efisien.

BAB V
PENUTUP

5.1 Kesimpulan

Praktikum ini menunjukkan bahwa kompleksitas algoritma mempengaruhi kecepatan dan efisiensi program. Algoritma dengan kompleksitas lebih rendah bekerja lebih cepat pada data besar. Hasil pengujian sesuai dengan teori Big O sehingga pemilihan algoritma yang tepat sangat penting.

5.2 Saran

Gunakan algoritma dengan kompleksitas rendah saat menangani data besar agar program lebih efisien. Hindari penggunaan nested loop jika tidak diperlukan. Manfaatkan struktur data seperti Set atau Map untuk meningkatkan performa. Lakukan pengujian dan pengukuran waktu eksekusi agar dapat memilih algoritma yang paling sesuai.

DAFTAR PUSTAKA

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. 2009. Introduction to Algorithms. MIT Press.

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser. 2014. Data Structures and Algorithms in Java. Wiley.

Eloquent JavaScript. 2018. Marijn Haverbeke. No Starch Press.

Mozilla Developer Network. JavaScript Guide. https://developer.mozilla.org

GeeksforGeeks. Time Complexity and Space Complexity. https://www.geeksforgeeks.org